Permutations and Operator Combinations

Generating arrangements in place

CSCI 235 – Data Structures

Hunter College, Fall 2026

Class 15 Thursday, October 22, 2026

Today

1. **Counting arrangements**: the dinner-party problem and $n!$.
2. **Generating** every arrangement with recursion: choose, explore, undo.
3. What goes wrong when you **forget the undo**.
4. **Operator combinations**: the same pattern, a different loop.
5. How the two pieces fit together in Make 24 (Project 3).

- **Exercise 7 (Permutation Trace)** is posted today and due **Tuesday, Nov 3**. Today's ideas are the toolbox for it; the handout has the exact tasks.
- Midterm: Tue, Oct 27, in class. Today's material is **not** on it.
- **Announcement:** the **midterm cheat sheet was corrected** on Oct 20. Please **re-download** it from the course schedule. Only the **provided** cheat sheet is allowed on the exam: no self-made sheets.

Question: The Dinner Party

Three guests, **A**, **B**, **C**, sit in three chairs in a row.

(a) List every seating arrangement. How many are there?
(b) Explain the count without listing: how many guests can take chair 1? Then chair 2? Then chair 3?
(c) How many arrangements for 4 guests? For 10 guests?

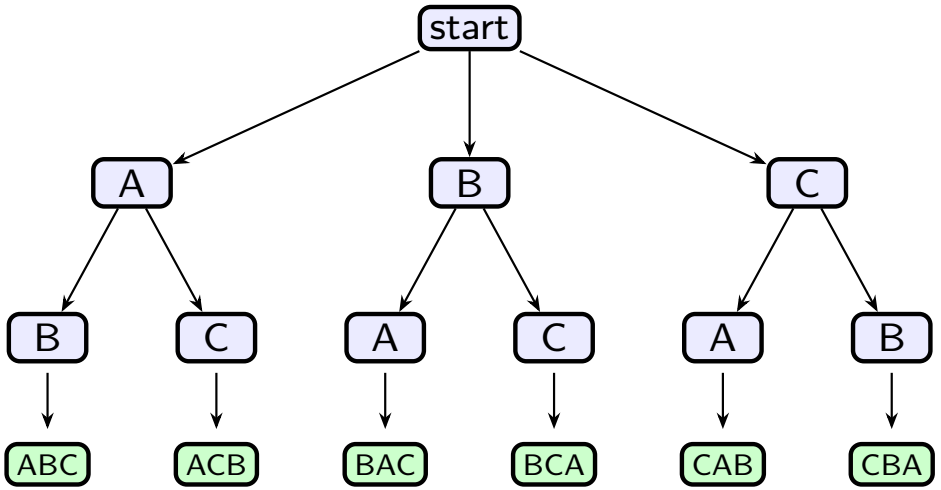
Question: The Dinner Party

Three guests, **A**, **B**, **C**, sit in three chairs in a row.

(a) List every seating arrangement. How many are there?
(b) Explain the count without listing: how many guests can take chair 1? Then chair 2? Then chair 3?
(c) How many arrangements for 4 guests? For 10 guests?

(a) ABC, ACB, BAC, BCA, CAB, CBA: **6**.
(b) Chair 1: 3 choices. Chair 2: 2 guests left. Chair 3: 1 guest left. $3 \times 2 \times 1 = 6$.
(c) $4! = 24$; $10! = 3\,628\,800$. In general $n! = n \times (n-1) \times \cdots \times 1$.

The Arrangement Tree



Read the tree **top to bottom**: each level is one chair.

Level 1: 3 branches. Level 2: 2 branches under each. Level 3: 1 branch.

$3 \times 2 \times 1 = 6$ leaves. This tree is the **state space**. A recursive function walks it **depth first**, visiting leaves left to right.

Generating Them: Swap, Recurse, Undo

Trick: keep the guests in one array. Chair `k` onward are *still unseated*. To fill chair `k`, swap each unseated guest into it.

```
void printAll(std::string& s, int k) {
    if (k == static_cast<int>(s.size())) {  // every chair filled
        std::cout << s << "\n";
        return;
    }
    for (int i = k; i < static_cast<int>(s.size()); ++i) {
        std::swap(s[k], s[i]);              // 1. choose
        printAll(s, k + 1);                 // 2. explore
        std::swap(s[k], s[i]);              // 3. undo
    }
}
// std::string s = "ABC"; printAll(s, 0);
```

Question: In what order are the six strings printed? (Trace by hand before the next slide.)

The Idea: Fill One Chair at a Time

Chairs $0, 1, 2$**.** To list every seating order:

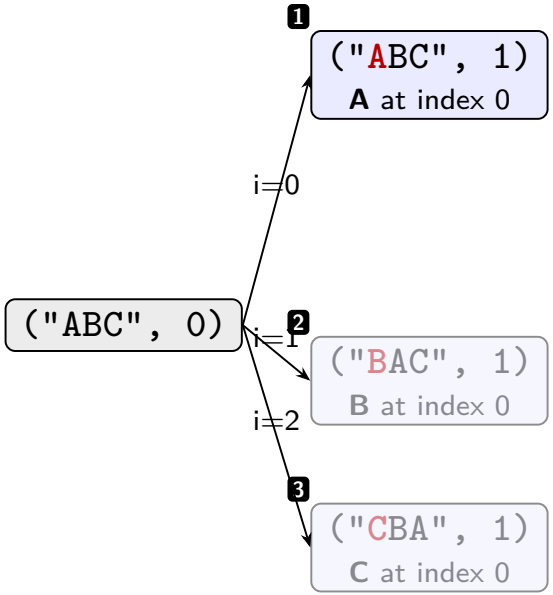
1. **Chair 0:** any guest may sit there. Swap the chosen guest into index 0.
2. **Chair 1:** now only the guests in positions $1, 2$ are still unseated. Swap one of them into index 1.
3. **Chair 2:** one guest is left, so there is no choice.
4. All `s.size()` chairs filled (`k` $==$ `s.size()`): **print** the string.

What `k` means: chairs $0, \ldots, k-1$ are already filled. Today's picture writes the string as `AB|C`: the part left of the bar is **fixed**, the part right of the bar is still **unseated**. The loop variable `i` says *which unseated guest* we try next.

Drawing the calls. Each box is one call `printAll(s, k)`. The black tag names it: **1, 2, 3** are the three choices for chair 0; **1-1, 1-2** are the two choices for chair 1 under call 1; and so on.

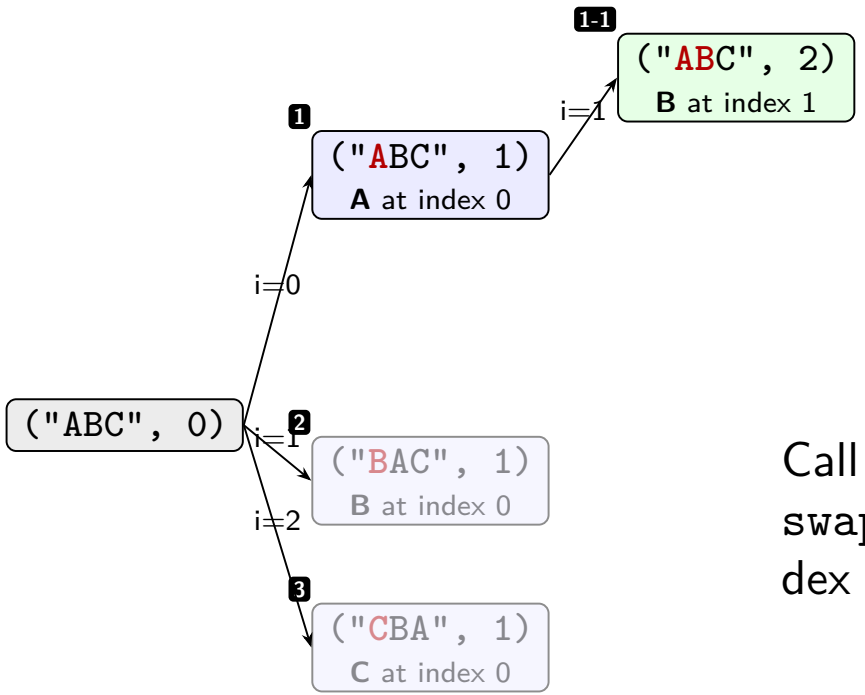
Following Branch 1 of `printAll("ABC", 0)`

$k = 0$ $k = 1$ $k = 2$ $k = 3$ $k = n$



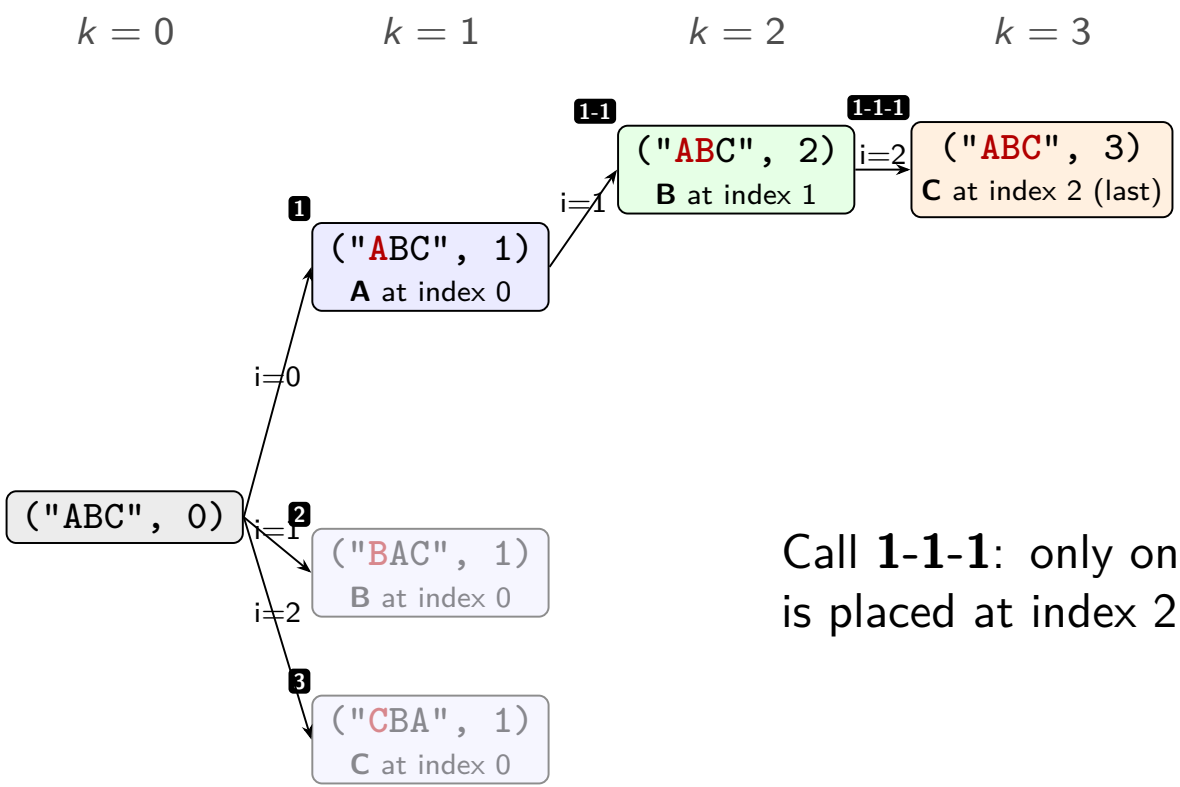
Chair 0 has three choices (`i=0,1,2`).
Call **1**: `swap(s[0],s[0])` changes nothing, so **A** is placed at index 0.

Following Branch 1 of `printAll("ABC", 0)`



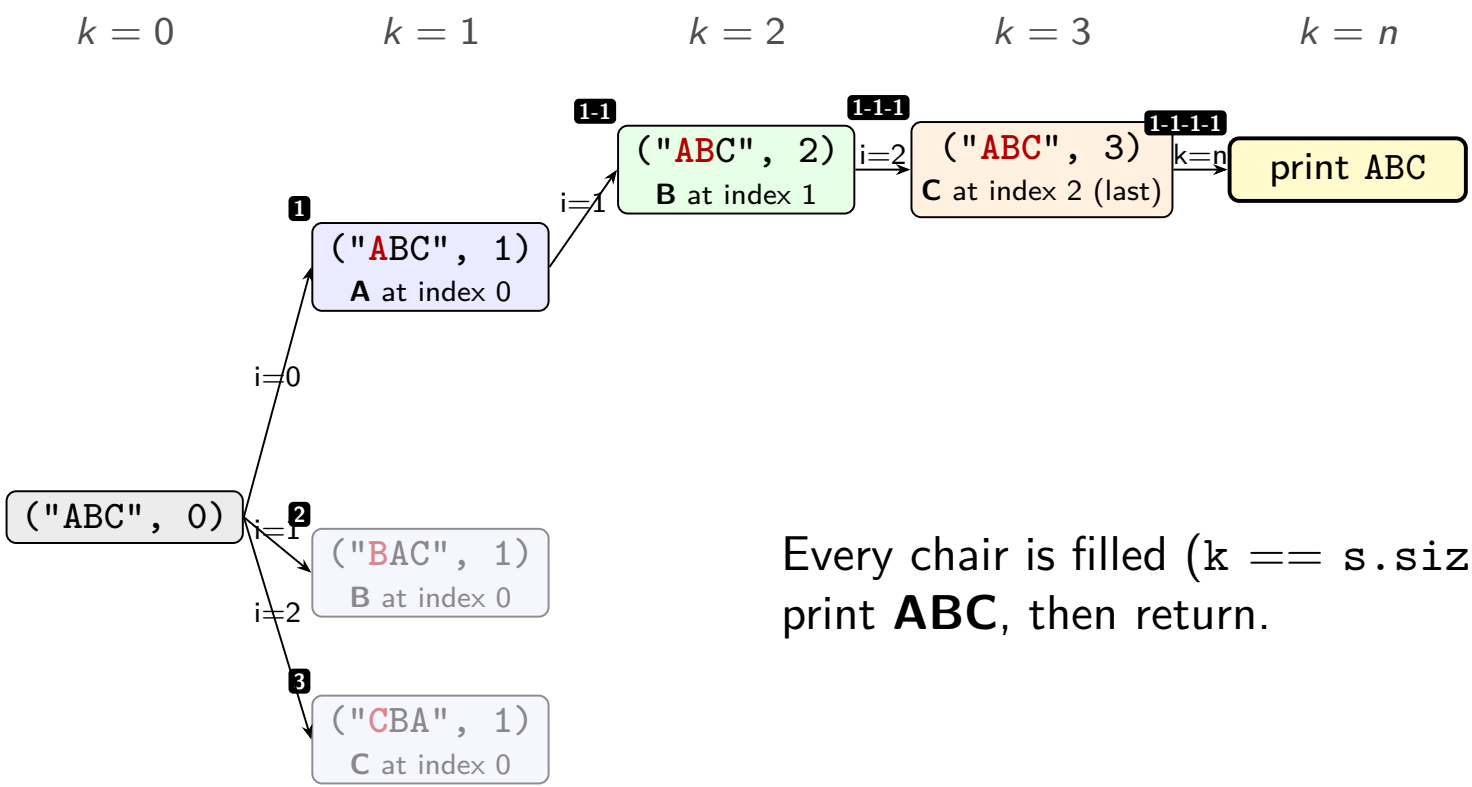
Call **1-1**: `i=1` means `swap(s[1],s[1])`. **B** is placed at index 1.

Following Branch 1 of `printAll("ABC", 0)`



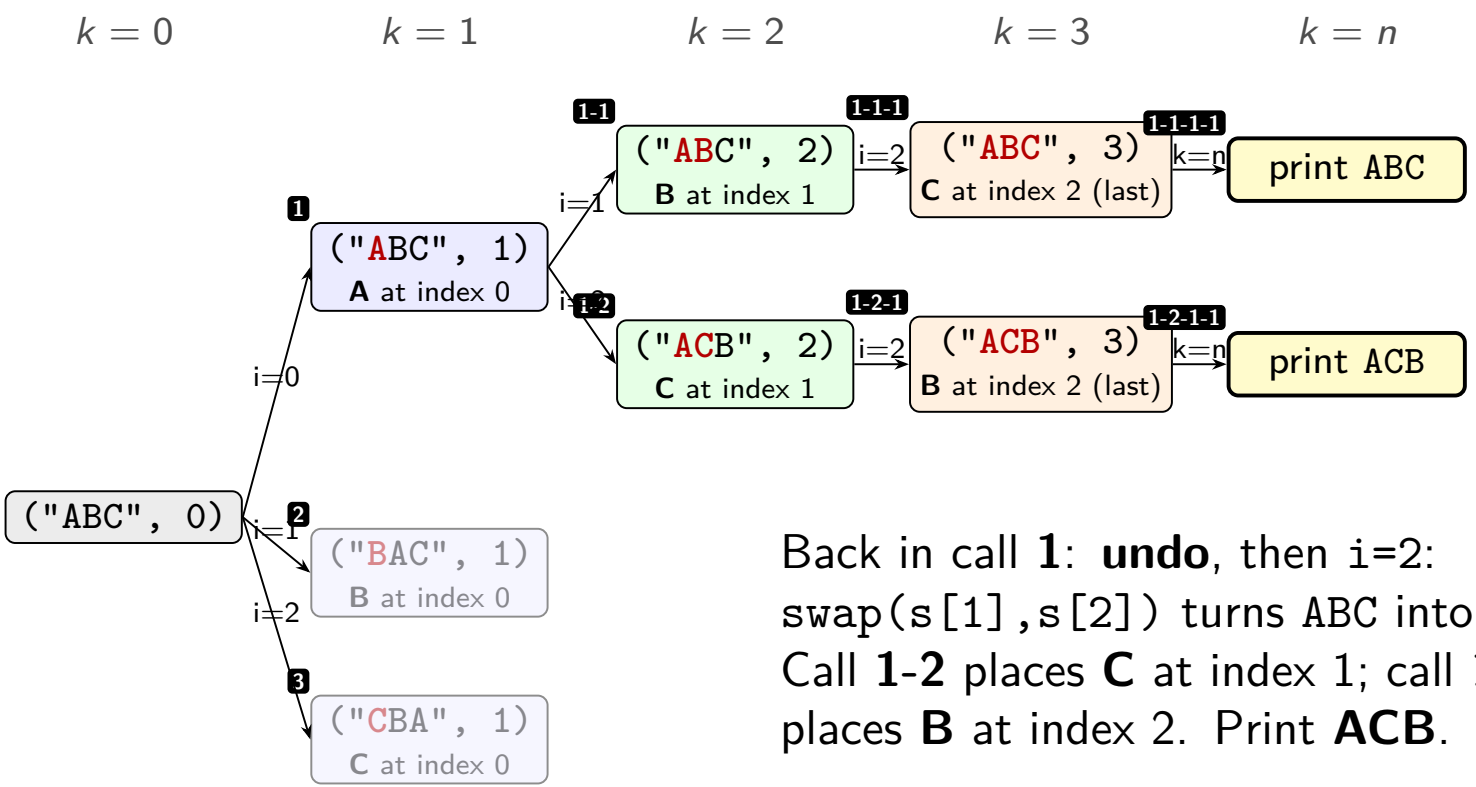
Call **1-1-1**: only one guest is left, so **C** is placed at index 2, the last position.

Following Branch 1 of `printAll("ABC", 0)`



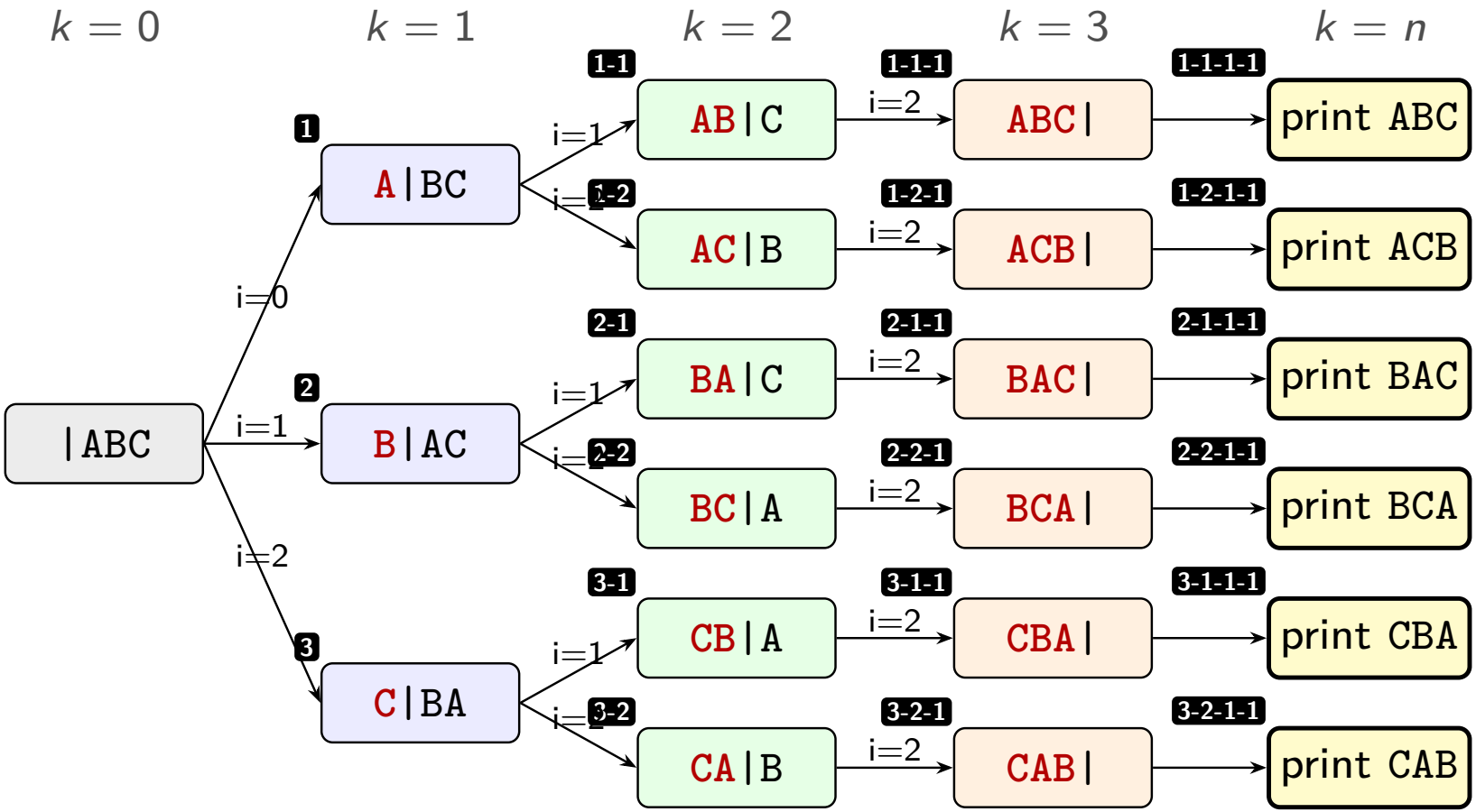
Every chair is filled (`k == s.size()`):
print **ABC**, then return.

Following Branch 1 of `printAll("ABC", 0)`



Back in call **1**: **undo**, then `i=2`: `swap(s[1],s[2])` turns `ABC` into `ACB`. Call **1-2** places **C** at index 1; call **1-2-1** places **B** at index 2. Print **ACB**.

The Full Call Tree for `printAll("ABC", 0)`



Red letters left of `|`: chairs already filled. `i`: the guest swapped in. Read the prints from the top down: ABC, ACB, BAC, BCA, CBA, CAB. ($1 + 3 + 6 + 6 = 16$ calls, then 6 prints.)

Question: Reading the Call Tree

Use the call tree for `printAll("ABC", 0)`.

(a) Which call places **C** at index 1 under call **2**? What is `s` when that call starts?
(b) How many calls are at each level $k = 0, 1, 2, 3$? Why does the number of calls stop growing after $k = 2$?
(c) For `n = 4`, how many strings are printed?

Question: Reading the Call Tree

Use the call tree for `printAll("ABC", 0)`.

(a) Which call places **C** at index 1 under call **2**? What is `s` when that call starts?
(b) How many calls are at each level $k = 0, 1, 2, 3$? Why does the number of calls stop growing after $k = 2$?
(c) For `n = 4`, how many strings are printed?

(a) Call **2-2**: `printAll("BCA", 2)`. Under call 2 (`s = "BAC"`), `i=2` swaps `s[1]` and `s[2]`.
(b) 1, 3, 6, 6. At $k = 2$ only one guest is left, so each call has just one child.
(c) $4! = 24$.

Tracing `printAll(s, 0)` with `s = "ABC"`

Step	call `(k)`, loop `i`	after swap	prints	after undo
1	k=0, i=0; then k=1, i=1	ABC		
2	k=1, i=2	ACB		
3	k=0, i=1; then k=1, i=1	BAC		
4	k=1, i=2	BCA		
5	k=0, i=2; then k=1, i=1	CBA		
6	k=1, i=2	CAB		

Tracing `printAll(s, 0)` with `s = "ABC"`

Step	call (k), loop i	after swap	prints	after undo
1	k=0, i=0; then k=1, i=1	ABC	ABC	ABC
2	k=1, i=2	ACB	ACB	ABC
3	k=0, i=1; then k=1, i=1	BAC	BAC	BAC
4	k=1, i=2	BCA	BCA	BAC
5	k=0, i=2; then k=1, i=1	CBA	CBA	CBA
6	k=1, i=2	CAB	CAB	CBA

Each time `k=0` finishes an iteration, the undo restores `ABC` before the next `i`. Output order: **ABC, ACB, BAC, BCA, CBA, CAB**. It differs from the tree order only in the last two leaves (the tree listed CAB before CBA).

Question: What If We Forget the Undo?

Remove the **second** `std::swap` (the undo). Same input `"ABC"`. What is printed?

(A) The same six strings, in the same order.

(B) Six strings, but some repeat and some arrangements are missing.

(C) Nothing is printed.

(D) The program crashes immediately.

Question: What If We Forget the Undo?

Remove the **second** `std::swap` (the undo). Same input `"ABC"`. What is printed?

(A) The same six strings, in the same order.

(B) Six strings, but some repeat and some arrangements are missing.

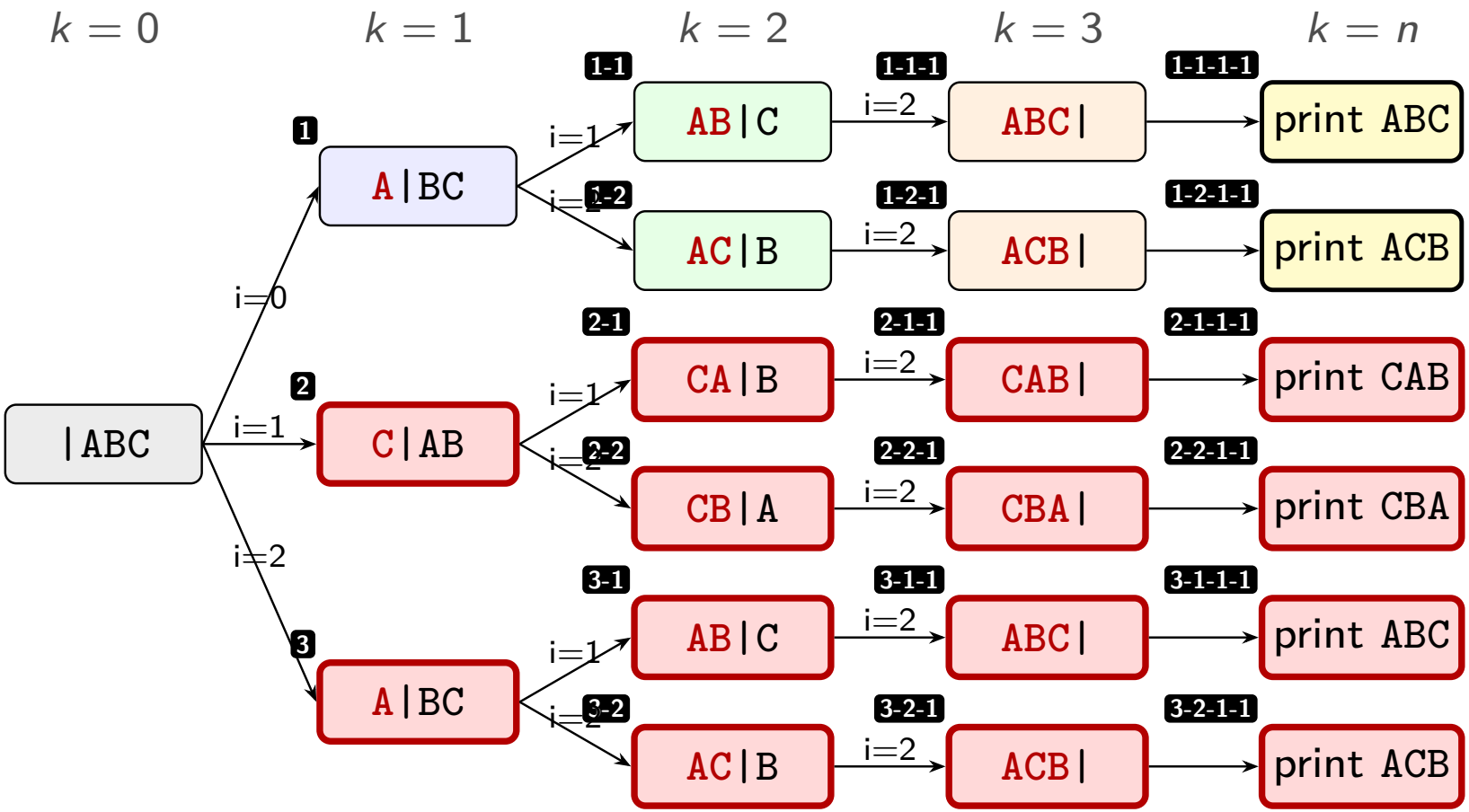
(C) Nothing is printed.

(D) The program crashes immediately.

Answer: (B). It prints **ABC, ACB, CAB, CBA, ABC, ACB**. Without the undo, the array is left in the *wrong* order after each recursive call, so the next `swap` starts from a scrambled state. Missing: BAC and BCA. Repeated: ABC and ACB.

Moral: choose and undo must **mirror** each other exactly.

Without the Undo: What the Call Tree Looks Like



Red = differs from the correct tree. Call **2** starts from `ACB` (left over from call 1), so its swap gives `CAB`, not `BAC`. BAC and BCA are never printed; ABC and ACB are printed twice.

Copying vs. In-Place

	Copy the state	**Change one state, then undo**
How	pass a new vector/string **by value** into each call	pass **by reference**; swap, recurse, swap back
Undo?	automatic: the copy is discarded	**you** must write it
Cost	copying at every call	two $O(1)$ swaps per branch; no string copy
Danger	slow for large states	forgetting or mis-ordering the undo

Analogy. Copying prints a fresh seating chart for every experiment; in-place erases and rewrites one chart: faster, but erase carefully.

By value protects the *caller* from a child's changes. It does **not** undo changes the caller makes to its own copy inside its loop.

Rule of thumb: copy while you learn; go in-place when the state is large or speed matters.

Duplicates: What About `"AAB"`?

Question. How many *distinct* arrangements does `"AAB"` have? How many strings does `printAll` print?

Duplicates: What About `"AAB"`?

Question. How many *distinct* arrangements does `"AAB"` have? How many strings does `printAll` print?

Distinct: **3** (AAB, ABA, BAA). `printAll` prints **6** lines:

AAB, ABA, AAB, ABA, BAA, BAA

The function treats the two A's as *different people*. Each distinct arrangement appears $2! = 2$ times.

Choices: accept the repeats (simple), filter them afterward (e.g. keep results in a `std::set`), or prune repeated branches while searching. Read your Exercise 7 handout to see what it asks for.

Operator Combinations: A Different Loop

Problem. Numbers 1 2 3 in order. Fill the **two gaps** with $+, -, *, /$. Print every expression.

```
void ops(const std::vector<int>& v, int k, std::string& expr) {
    if (k == static_cast<int>(v.size())) { std::cout << expr << "\n";
        return; }
    for (char op : std::string("+-*/")) {
        std::size_t len = expr.size();          // remember the length
        expr += op;                             // 1. choose
        expr += std::to_string(v[k]);
        ops(v, k + 1, expr);                    // 2. explore
        expr.resize(len);                       // 3. undo
    }
}
// std::string e = "1"; ops({1,2,3}, 1, e);
```

Note: it prints expression *strings* only: no evaluation, parentheses, or division-by-zero check.

Question: How many lines? What are the first four?

Operator Combinations: A Different Loop

Problem. Numbers 1 2 3 in order. Fill the **two gaps** with $+, -, *, /$. Print every expression.

```
void ops(const std::vector<int>& v, int k, std::string& expr) {
    if (k == static_cast<int>(v.size())) { std::cout << expr << "\n";
        return; }
    for (char op : std::string("+-*/")) {
        std::size_t len = expr.size();          // remember the length
        expr += op;                             // 1. choose
        expr += std::to_string(v[k]);
        ops(v, k + 1, expr);                    // 2. explore
        expr.resize(len);                       // 3. undo
    }
}
// std::string e = "1"; ops({1,2,3}, 1, e);
```

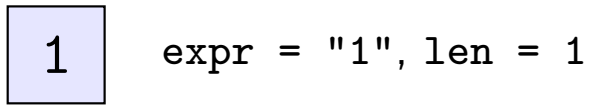
Note: it prints expression *strings* only: no evaluation, parentheses, or division-by-zero check.

Question: How many lines? What are the first four?

16 lines. First four: 1+2+3, 1+2−3, 1+2*3, 1+2/3; the **last** gap changes fastest.

What Does `expr.resize(len)` Mean?

`s.resize(n)` sets the **length** of the string `s` to `n`. When `n` is smaller than the current length, it **cuts characters off the end** and keeps the first `n`. (When `n` is larger, it pads; we never do that here.)

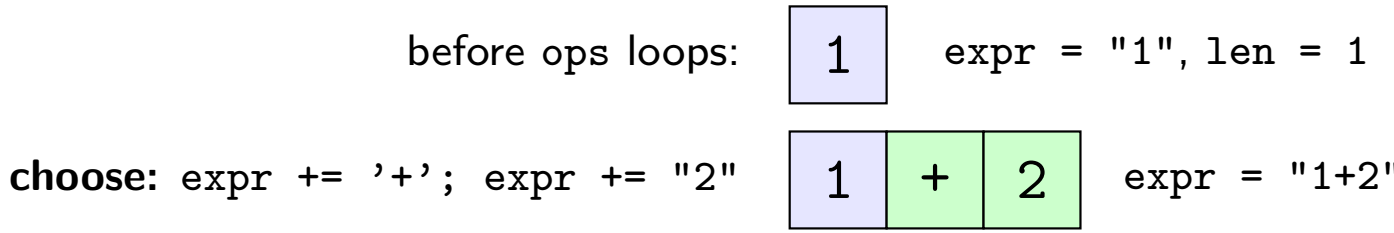


Why remember `len`? Choose **appends** an operator and a number, and `to_string(v[k])` may be several characters (e.g. `"12"`), so a fixed number of `pop_back()` calls is not sufficient in general, since the appended number may contain multiple characters. Save `len = expr.size()` **before** choosing; the undo cuts the string back to that length, whatever was appended.

Compare with permutations: there, undo was *the same swap again*. Here, undo is *resize to the saved length*. Either way the undo **mirrors** the choose, so the shared object looks as it did before.

What Does `expr.resize(len)` Mean?

`s.resize(n)` sets the **length** of the string `s` to `n`. When `n` is smaller than the current length, it **cuts characters off the end** and keeps the first `n`. (When `n` is larger, it pads; we never do that here.)

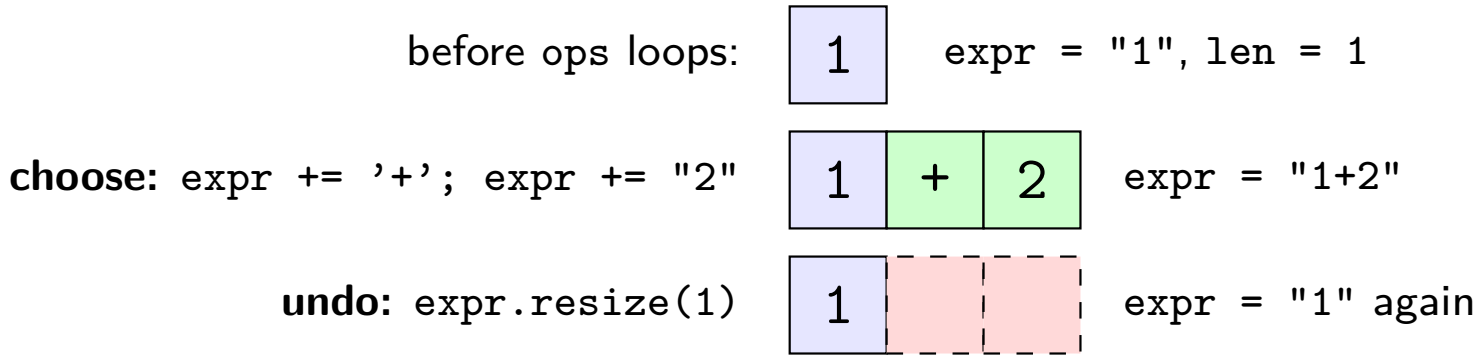


Why remember `len`? Choose **appends** an operator and a number, and `to_string(v[k])` may be several characters (e.g. `"12"`), so a fixed number of `pop_back()` calls is not sufficient in general, since the appended number may contain multiple characters. Save `len = expr.size()` **before** choosing; the undo cuts the string back to that length, whatever was appended.

Compare with permutations: there, undo was *the same swap again*. Here, undo is *resize to the saved length*. Either way the undo **mirrors** the choose, so the shared object looks as it did before.

What Does `expr.resize(len)` Mean?

`s.resize(n)` sets the **length** of the string `s` to `n`. When `n` is smaller than the current length, it **cuts characters off the end** and keeps the first `n`. (When `n` is larger, it pads; we never do that here.)



Why remember `len`? Choose **appends** an operator and a number, and `to_string(v[k])` may be several characters (e.g. `"12"`), so a fixed number of `pop_back()` calls is not sufficient in general, since the appended number may contain multiple characters. Save `len = expr.size()` **before** choosing; the undo cuts the string back to that length, whatever was appended.

Compare with permutations: there, undo was *the same swap again*. Here, undo is *resize to the saved length*. Either way the undo **mirrors** the choose, so the shared object looks as it did before.

Trace: `ops({1,2,3}, 1, e)` with `e = "1"`

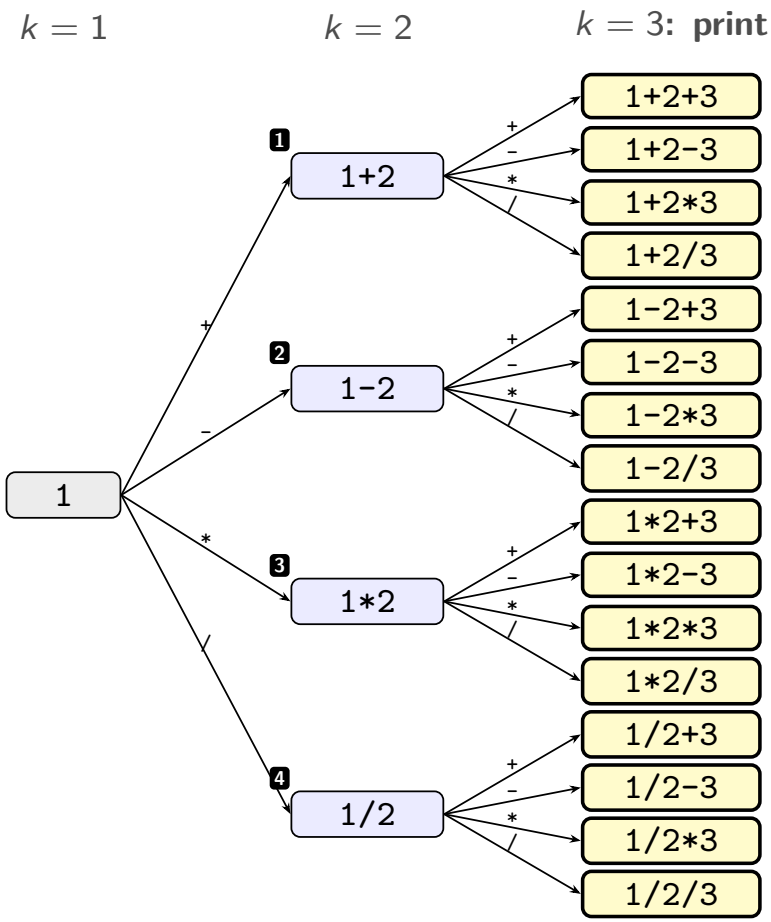
Step	call	`len`	action	`expr` after
1	`ops(v,1,"1")`	1	choose +2	`1+2`
2	`ops(v,2,"1+2")`	3	choose +3	`1+2+3`
3	`ops(v,3,"1+2+3")`	—	$k = 3$: **print `1+2+3`**, return	`1+2+3`
4	back in call 2	3	`resize(3)`, then choose `-3`	`1+2-3`
5	`ops(v,3,"1+2-3")`	—	**print `1+2-3`**, return	`1+2-3`
6	back in call 2	3	`resize(3)`, choose `*3`, print	`1+2*3`
7	back in call 2	3	`resize(3)`, choose `/3`, print	`1+2/3`
8	back in call 2	3	`resize(3)`: loop over; return	`1+2`
9	back in call 1	1	`resize(1)`, then choose `-2`	`1-2`
	... the same four lines for `1-2`, `1*2`, `1/2` ...			
last	back in call 1	1	`resize(1)`: loop over; return	`1`

Trace: `ops({1,2,3}, 1, e)` with `e = "1"`

Step	call	`len`	action	`expr` after
1	`ops(v,1,"1")`	1	choose +2	`1+2`
2	`ops(v,2,"1+2")`	3	choose +3	`1+2+3`
3	`ops(v,3,"1+2+3")`	—	$k = 3$: **print 1+2+3**, return	`1+2+3`
4	back in call 2	3	`resize(3)`, then choose -3	`1+2-3`
5	`ops(v,3,"1+2-3")`	—	**print 1+2-3**, return	`1+2-3`
6	back in call 2	3	`resize(3)`, choose *3, print	`1+2*3`
7	back in call 2	3	`resize(3)`, choose /3, print	`1+2/3`
8	back in call 2	3	`resize(3)`: loop over; return	`1+2`
9	back in call 1	1	`resize(1)`, then choose -2	`1-2`
	... the same four lines for `1-2`, `1*2`, `1/2` ...			
last	back in call 1	1	`resize(1)`: loop over; return	`1`

Result: 16 lines (4 choices at $k = 1$, times 4 at $k = 2$). At the end `e` is `"1"` again, exactly as passed in.

Call Tree of `ops({1,2,3}, 1, e)`



Each call tries the four operators in the order + − ∗ /. Left to right = the order of the 16 printed lines. $1 + 4$ calls reach $k = 2$ or below, then 16 calls reach $k = 3$ and print.

Question: `ops` (Today) vs. `search` (Class 14)

Class 14's `search(v, k, value, target)` also has 4 branches (`+ - * /`) at each of $k = 1, 2$: the **same tree shape** as the call tree for `ops`. What is **different**?

Question: `ops` (Today) vs. `search` (Class 14)

Class 14's `search(v, k, value, target)` also has 4 branches (`+ - * /`) at each of $k = 1, 2$: the **same tree shape** as the call tree for `ops`. What is **different**?

	`search` **(Class 14)**	`ops` **(today)**
Goal	Can we reach `target`? Returns `bool`.	List every expression. Returns `void`.
Carries	A number `value`, evaluated left to right.	A string `expr`, never evaluated.
Passed by	**Value**: each call gets its own copy.	**Reference** (`string&`): all calls share one string.
Undo?	**None needed**: nothing shared is changed.	**Required**: `expr.resize(len)`.
Stops early?	Yes: `||` stops at the first `true`.	No: it prints all 16 lines.

Question: `ops` (Today) vs. `search` (Class 14)

Class 14's `search(v, k, value, target)` also has 4 branches (`+ - * /`) at each of $k = 1, 2$: the **same tree shape** as the call tree for `ops`. What is **different**?

	`search` **(Class 14)**	`ops` **(today)**
Goal	Can we reach `target`? Returns `bool`.	List every expression. Returns `void`.
Carries	A number `value`, evaluated left to right.	A string `expr`, never evaluated.
Passed by	**Value**: each call gets its own copy.	**Reference** (`string&`): all calls share one string.
Undo?	**None needed**: nothing shared is changed.	**Required**: `expr.resize(len)`.
Stops early?	Yes: `||` stops at the first `true`.	No: it prints all 16 lines.

Same path, two states $(1 + 2 + 3)$: `search` carries $1 \to 3 \to 6$; `ops` carries `"1"`$\to$`"1+2"`$\to$`"1+2+3"`.

Moral: same tree; the difference is the **state** and whether it is **shared**. A copied value needs no undo; a shared string must be restored.

Putting the Counts Together

A Make 24 solver that tries **every** order ($n!$), every operator choice (4^{n-1}), and every parenthesization (Catalan numbers 2, 5, 42):

n	$n!$	4^{n-1}	parenth.	**total**
3	6	16	2	
4	24	64	5	
6	720	1 024	42	

Question: Fill the last column: multiply the three columns.

Putting the Counts Together

A Make 24 solver that tries **every** order ($n!$), every operator choice (4^{n-1}), and every parenthesization (Catalan numbers 2, 5, 42):

n	$n!$	4^{n-1}	parenth.	**total**
3	6	16	2	192
4	24	64	5	7 680
6	720	1 024	42	30 965 760

Question: Fill the last column: multiply the three columns.

The total counts enumeration choices (orders $\times$ operators $\times$ parenthesizations), not necessarily distinct expressions or distinct results. It is a *different enumeration* from last class's 3888 (counted before excluding division by zero; that pair search used one orientation for $+$ and $\times$). With repeated values, $n!$ counts input *positions*. Six numbers mean tens of millions of checks: we will be clever on Oct 29.

Common Bugs

- **Undo that does not match the choice** (wrong indexes, wrong order, or missing).
- Passing the state **by value** when you meant by reference (changes vanish), or by reference when you needed a copy (changes leak between branches).
- Looping from 0 instead of from `k`: already-seated guests get swapped again.
- **Printing or recording the result at the wrong depth**: do it only in the base case.
- Forgetting that results from the loop come out in **depth-first order**, not sorted order.
- Storing a *reference* to a changing state instead of a **copy** of the finished result.

Practice

1. How many arrangements of 5 different letters?
2. In `printAll("ABCD", 0)`, how many lines are printed? How many times is the base case reached?
3. In `printAll(s, 0)` with `s = "ABC"`, the call with `k = 1` runs its loop for `i` $= 1$ and 2. How many values does `i` take when `k = 2`?
4. How many expressions does `ops` print for 5 numbers?

Practice

1. How many arrangements of 5 different letters?
2. In `printAll("ABCD", 0)`, how many lines are printed? How many times is the base case reached?
3. In `printAll(s, 0)` with `s = "ABC"`, the call with `k = 1` runs its loop for `i` = 1 and 2. How many values does `i` take when `k = 2`?
4. How many expressions does `ops` print for 5 numbers?

1. $5! = 120$.
2. $4! = 24$ lines; the base case is reached 24 times, once per leaf.
3. One: $i = 2$ only. A single chair left has a single guest.
4. $4^4 = 256$.

Summary and Preview

- *n distinct* items have $n!$ arrangements; the search tree has one level per chair.
- Recursive enumeration: **choose, explore, undo**, and the undo mirrors the choose.
- Operator combinations use the same pattern, with 4 choices per gap.
- The counts multiply: orders $\times$ operators $\times$ parentheses.

Coming up

Oct 27: Midterm (through DFS/BFS).
Oct 29: pruning and early termination: how to avoid walking the whole tree.
Nov 3: Exercise 7 due.